

MANTIS presentation — read-aloud script

Deck: MANTIS_Presentation_2026-09-24.pptx · 18 slides · about 21 minutes at a relaxed pace.

Read each block as written; it is in spoken language. The same text is in the speaker notes of each slide. If you are short on time, skip slides 8, 16 and 17 and shorten slides 4–5 to one sentence each.

Slide 1 — MANTIS — features, architecture, attacks, and results

About 1 min

Good morning, everyone, and thank you for having us. Today I'm going to walk you through MANTIS. I'll start with what it is and everything that has been built into it so far. Then I'll show you how it is put together, the attacks and faults you can run against it, what happened when we ran them, and the defenses we've built on the same framework. And I'll finish with what I think matters most for adoption: how easy it is for someone new to run it and add an attack of their own. The one-line summary is on the screen. MANTIS lets you attack a bank of AI agents on purpose, and then know exactly what happened. Everything I'll show comes from real runs against a realistic banking system that uses synthetic data only.

Slide 2 — What MANTIS is

About 1.25 min

So what is MANTIS? It is a testbed for multi-agent AI systems in banking. Banks are starting to split work across specialist AI agents: one watches transactions, one screens for fraud, one checks policy, one makes the decision. That is powerful, and it also means every hand-off between agents, every message, and every tool call is a new place where something can be attacked, or can simply fail. MANTIS gives you a way to test that on purpose, and to measure the result. There are three ideas behind it. One: the system under test is a real banking multi-agent system, and it is never modified. Two: an experiment is just a configuration file. You declare an attack, a fault, or a defense in YAML and it plugs in. Three: every run produces evidence and an automatic verdict, with enough recorded to reproduce it. And the boundary, stated plainly: MANTIS is a testbed. It measures what an attack or a fault did. It is not a detector, and it is not a defense product.

Slide 3 — Architecture: five control points, one hook bus

About 1.25 min

The architecture is deliberately simple. Everything an agent does passes one of five checkpoints: when a request arrives, when an agent starts, when an agent talks to the model, when a tool is about to act, and when the final result leaves. Those checkpoints sit on what we call the hook bus. Any plugin can register on the bus and see, and optionally change,

what is passing through. There are four kinds of plugin. Attacks inject a fault. Failures simulate ordinary faults such as a delay or a timeout. Defenses are guardrails that deny, redirect, or redact. And the observability plugin, which records everything and always runs last, so it sees what every earlier plugin did. At a checkpoint a plugin can do one of six things: continue, mutate, deny, error, skip, or delay. The consequence that matters is that we never modify the banking code to run an experiment.

Slide 4 — What has been built: the foundation

About 1.5 min

Here is what has been built, starting from the beginning. The foundation was delivered as nine work packages. We started by freezing and characterizing the original banking system: forty-nine regression tests against recorded golden runs, so the system under test stays the same. Next we modularized it behind a runtime adapter that reports the live inventory of agents and tools. Then configuration: experiments are YAML, checked against a schema and against the live system, and every configuration is hashed. The hook bus and the five control points came next; that is the heart of the architecture. On top of it, the observability pipeline: JSONL traces, OpenTelemetry, MLflow, and a manifest for every run. Then the first attack and fault plugins: prompt injection, message spoofing, route confusion, tool mutation, and reliability faults. Then evaluation and benchmarking: seven scored dimensions, and a measured instrumentation overhead of about point-two percent. Then the command-line workflow, including campaigns that sweep a whole folder. And finally the tests, documentation, and an open-source release.

Slide 5 — What has been built since: the extensions

About 1.5 min

Since the foundation, we've extended the testbed in six directions, all on the same plugin interface. First, exporters: traces can go to Jaeger, Grafana Tempo, Langfuse, and Phoenix, in addition to OpenTelemetry and MLflow, each turned on with one configuration line. Second, more banking workloads: dispute filing, suspicious-activity escalation, and loan pre-approval, each backed by real persisted records; the loan decision is a deterministic rule, not an opinion. Third, a second institution: a regional credit union that runs the same agents against its own data and a stricter threshold, so the same request gets a genuinely different decision. Fourth, six defenses that cover every category of security mechanism in the plan: guardrails, policy checks, response filters, rate limits, and isolation. Fifth, a small user interface: a config editor, a trace viewer, and campaign reports. And sixth, an extended scenario library: nineteen measured baselines and twenty-six attack variants, each run with database-level evidence. Every run now also records which model served it.

Slide 6 — The system under test

About 1 min

This is what we test against. Every request enters through a root agent, which routes it to a front-office, mid-office, or back-office router. The front office is customer-facing: transaction monitoring with fraud and compliance review, a customer-service chatbot, and dispute filing. The mid office is internal: operations planning and staffing, representative assist, and loan pre-approval. The back office is end-of-day reconciliation and reporting, and suspicious-activity escalation. In total that is thirty-four agents and twenty-seven tools, with two institution profiles. It sits on a real banking service and database, so when a transfer executes or a report is filed, real state changes. That is what makes the results meaningful. And all of the data is synthetic.

Slide 7 — An experiment is a YAML file

About 1 min

This is a complete experiment. It names the scenario, the attack plugin, where it attaches, and what a correct run looks like. This one is route confusion: it hijacks the routing decision, so when the router hands a suspicious transaction to fraud and compliance review, the request is quietly sent to the ordinary customer-service workflow instead. The evaluation block at the bottom is the recorded baseline for this scenario, and that is what the scorecard compares against. Four commands drive everything. Validate checks every name against the live system before anything runs. Run executes real agents against a real backend and a live model. Evaluate scores the trace against the baseline. And campaign sweeps a whole folder of configurations into one comparative report.

Slide 8 — What every run produces

About 1.25 min

After every run you get a folder of evidence. The manifest fingerprints the configuration, the seed, the environment, and which model served the run, so any result can be reproduced. The trace is a timestamped record of every agent, message, tool call, and plugin action. Hook coverage records which checkpoints fired, so an attack that could not reach its target is never mistaken for a working one. And the evaluation scores seven dimensions: completeness, tool use, outcome, hook coverage, domain coverage, integrity, and attack ground truth. The idea I most want you to take from this slide is at the bottom: fired is not the same as succeeded. In this example, the attack rewrote a small transfer into five thousand dollars to a fake account. That is fired. The bank's own account validation rejected it and no money moved. That is no effect. We report those as two separate answers, and the trace uses three distinct event types, so an attack, a defense, and an ordinary fault are never confused with each other.

Slide 9 — The attack and fault catalog

About 1 min

Here is what you can run against the system. Prompt injection adds an instruction to an

agent’s request to the model. Message spoofing forges a message from a trusted colleague agent, for example a fake compliance clearance. Route confusion hijacks a hand-off between agents. Tool-parameter mutation rewrites the arguments of a real action just before it runs. And reliability faults simulate a delay, a timeout, or a corrupted result. The first four are attacks; the last is ordinary failure, and we keep the two separate on purpose. On top of these original five families we built an extended library: 26 variants, eight injection payload techniques, and coverage that reaches the mid office and back office as well as the front office.

Slide 10 — Results: the original five attack families

About 1 min

These are the original five families, each run five times against the live system. Green means the system held; red means the attack got through. Prompt injection and message spoofing reached the model’s real request every time, and the model resisted. The funds-transfer mutation fired every time, and the bank’s own account validation stopped it. Two results stand out. Route confusion diverted a suspicious transaction away from compliance review every single time, with no defense in place at that stage. And the back-office ledger attack posted to the wrong batch without anything downstream noticing. Reliability faults were handled safely and logged as ordinary anomalies rather than attacks. All of these used gpt-oss-20b (open-weight), served through UF Navigator.

Slide 11 — Defenses run through the same framework

About 1.5 min

Because attacks are measurable, defenses use exactly the same mechanism, so you can compare them on equal footing. We have built six, and together they cover all five categories of security mechanism in the plan: guardrails, policy checks, response filters, rate limits, and isolation. The design rule is that a defense re-checks backend truth rather than trusting what the model decided. The amount-limit guardrail denies transfers above a limit. The routing guard re-checks the transaction’s real risk score and, in redirect mode, sends the request back to the compliant path, so the run still ends in manual review. The batch integrity guard blocks ledger posts against a batch with an unresolved discrepancy. Response redaction masks account numbers in the final answer. The rate limit contains runaway hand-off loops. And isolation is a shadow mode: nothing that moves money can execute, whatever an attack rewrote. For scope: this is coverage of the gaps we measured, and a foundation for evaluating your own guardrails, not a comprehensive defense suite.

Slide 12 — Results: the extended scenario library

About 1 min

To widen coverage we built a scenario library on the same architecture: 19 measured baselines and 26 attack and fault variants. Ground truth is measured, not assumed. Each baseline was run repeatedly, and each attack is scored against the recorded behavior of its scenario.

Every trial also starts from a freshly seeded database and records what changed afterwards, because some attacks leave no trace in tool use and show up only in the data. The table shows results by family across 123 scored live trials. Please read it as what happened in this sample, not as a rate to extrapolate. One configuration, the root hand-off hijack, halts the run on every attempt, so it has no score; the next slide covers it alongside the other findings.

Slide 13 — What the testbed revealed

About 1.75 min

These are four things the extended library revealed, each confirmed against the recorded traces and database snapshots. First, where an attack lands matters. The same authority-impersonation payload changed the outcome in 1 of 5 trials when aimed at the upstream monitor, but bypassed manual review in 5 of 5 when aimed at the decision agent. Second, some attacks succeed by making an agent do nothing. When injected, the suspicious-activity agent never looked up the case and filed nothing, 5 of 5 times, and a forged ‘closed as routine’ message suppressed the filing in 4 of 5. Third, database snapshots reveal effects that tool-use scoring alone would call clean. In 20 of 20 trials the database changed: a loan filed under the wrong customer, an inflated amount that flipped a decision, a report on the wrong case, a report under the wrong batch. Tool-use scoring flagged 0 of them. That is why every trial records the database. Fourth, faults propagate. A corrupted response from the ledger tool halted the end-of-day workflow in 5 of 9 attempts, and hijacking the root hand-off halted it 10 of 10 times. That is exactly the kind of behavior the testbed exists to expose. The scope note at the bottom: five trials per configuration and one model.

Slide 14 — Getting started: from clone to first experiment

About 1 min

Now the part I think matters most for adoption: how easy it is to get started. It is five steps. One, clone the repository and install it with a single pip command. Two, add a model key to the environment file. Or skip that entirely: mock mode runs the whole pipeline against a mock model, so a newcomer can try MANTIS with no key and no cost. Three, start the banking backend with one command. Four, run mantis inventory, which lists every agent, tool, and domain the platform sees, and confirms the installation. Five, run your first experiment with one command. Every command is written out in the guide folder in the repository. And there is nothing to configure inside the banking code. You are working with configuration files and, if you want, a small plugin.

Slide 15 — Adding your own attack

About 1.25 min

And this is how a new person adds their own attack. There are two paths. The first is configuration only, and it takes about two minutes. You copy an existing configuration, change the target and the parameters, and run validate, run, and evaluate. That covers

most new experiments, because the plugins are parameterized. The second path is a brand-new attack plugin, and it is about fifteen lines. You give it a name, say which control point it acts on, and write one function, apply, that looks at what is passing through and returns continue, mutate, deny, and so on. You register it with two lines, point a YAML file at it, and run. I tried this from scratch to make sure I am not overselling it. A new plugin, registered, configured, validated, run, and scored took about ten seconds in mock mode, with no API key, and the trace recorded the attack event correctly. Either way, you never touch the banking agents.

Slide 16 — See it run

About 1 min

Now a short live demo: how to run one of the built-in attacks and see what it did. It is five steps. Step one, open the project and activate the environment. Step two, run inventory. It introspects the live system and lists the agents, tools, and domains, so we know MANTIS sees the bank. Step three, run the route-confusion attack. This starts the real banking agents against the real backend and a live model, and injects the attack. Step four, evaluate the run. This scores the trace against the recorded baseline: did the attack fire, and did the outcome change. Step five, open the trace viewer in the browser. Every run appears in the list. I open this one and you can see the attack event in the trace, the scorecard, and which checkpoints were reached. In this run the attack fired and the workflow ended completed where manual review was expected, so the scorecard flags the diverted outcome. If the model is slow, I have this run recorded and ready to open, so the demo does not depend on a live call.

Slide 17 — Scope today, and where it goes next

About 1 min

To close the technical part, here is the scope today and where it goes next. Today, ground truth is a declared baseline plus each plugin's own report, so MANTIS is a measurement instrument rather than an independent detector. Results use five trials per configuration and one model, gpt-oss-20b (open-weight), served through UF Navigator. The data and backend are synthetic and sandboxed. Scoring covers actions and outcomes, with database snapshots for data-level effects. And the defenses cover the gaps we measured. Where it goes next follows naturally from that. The main direction is more scenarios: we already grew the library to nineteen baselines and twenty-six attack variants without changing the architecture, and the same recipe, a new prompt or config over the existing agents, tools and seeded data, adds more workflows, more attack variants, and more measured baselines. Then larger campaigns along the same axes to tighten the numbers. More models, so results can be compared across model families. More institution profiles and workflow patterns. And a place to evaluate other people's guardrails through the same framework, against the same attacks.

Slide 18 — Summary and questions

About 1 min

To summarize. MANTIS gives you a real banking multi-agent system to test against. It gives you five control points where anything can be observed or changed. Attacks, faults, and defenses are all plugins, declared in YAML. Automatic scoring separates whether something fired from whether it succeeded, and every run leaves evidence you can reproduce. A new person can be running an experiment within minutes and adding an attack of their own within an afternoon, and no banking code changes. The code and the step-by-step guide are in the repository. I would be glad to take your questions.

Appendix: demo commands (copy and paste)

Paste this whole block into one terminal. It starts the backend if it is not already running, runs the route-confusion attack under a `demo_` name (so recorded paper evidence is never overwritten), scores it, and opens the trace viewer.

```
cd ~/Desktop/MANTIS && source .venv/bin/activate
curl -s http://127.0.0.1:8000/health > /dev/null || (nohup python -m uvicorn app.main:ap
sleep 5 && curl -s http://127.0.0.1:8000/health
mantis --inventory
sed 's/name: wp5_route_confusion/name: demo_route_confusion/' configs/attacks/wp5_route_
mantis --run /tmp/demo_route_confusion.yaml
mantis --evaluate run_artifacts/demo_route_confusion
(open http://127.0.0.1:8765 &) ; mantis --ui
```

In the browser: pick `demo_route_confusion` in the Run dropdown, then **Load trace** (look for the `ATTACK_INJECTED` row), **View evaluation** (`attack_ground_truth`, `workflow_outcome`) and **View hook coverage**. Stop the viewer with Ctrl+C.

If the model is slow or the network is blocked, skip the run and pick an existing run such as `wp5_route_confusion` in the dropdown.